# Go 面试八股笔记

面向 Go 后端 / AI Agent 开发实习

**Go 面试八股笔记**

一、Go 基础

1. Go 语言优势与协程
2. make 与 new、数组与切片
3. for range 的坑
4. 字符串、rune 与拼接
5. defer、panic 与 recover
6. 空结构体、tag、init 与格式化
7. 函数传参、多返回值与 nil 比较
8. unsafe.Pointer 与 uintptr

二、slice 与 map

9. slice 底层结构与扩容
10. slice 共享底层数组的陷阱
11. map 底层实现（经典 hmap 结构，Go 1.24 之前）
12. map 扩容
13. map 使用要点

三、channel 与 select

14. CSP 与 channel 底层结构
15. 发送与接收流程
16. channel 操作结果总结
17. channel 导致的 goroutine 泄漏
18. select 机制与实现

四、sync 与并发原语

19. 安全读写共享变量的方式
20. 原子操作与锁
21. Mutex 实现与两种模式
22. sync.Once 与 WaitGroup
23. sync.Map

五、context

24. context 的作用与接口
25. 取消传播与 Value 查找

六、interface 与反射

26. interface 底层：eface 与 iface
27. 类型转换与类型断言

28. 反射
七、GMP 调度模型
29. GMP 模型
30. 调度策略
31. 调度时机与查找可运行 G 的顺序
32. 为什么需要 P、g0 与 m0
八、内存管理与 GC
33. 内存分配
34. 内存逃逸
35. 内存泄漏场景与排查
36. 三色标记法与 GC 根对象
37. 写屏障与混合写屏障
38. GC 流程、触发时机与调优
九、常见并发模式与代码题
39. 常见并发模式
40. 并发控制 + 超时（代码模板）
41. 交替打印（代码模板）

> 使用说明：按「主题 → 知识点」组织的结构化笔记，知识框架主要参考小林coding 面试题合集（Go），已重新整理改写。先记加粗关键词，再展开。涉及版本差异处已标注版本号。
> Java 基础与集合见 Java 面试八股笔记。

# Go 面试八股笔记

## 一、Go 基础

### 1. Go 语言优势与协程

- 优势：**语法简洁**、编译快、**静态编译单一二进制**部署方便、**原生并发（goroutine + channel）**、自带 GC、标准库强大（net/http）、工具链完善（fmt、test、pprof、race）。
- **进程**：资源分配的基本单位，独立地址空间；**线程**：CPU 调度的基本单位，共享进程资源，由内核调度，栈通常 MB 级；**协程（goroutine）**：用户态轻量级线程，由 **Go runtime 调度**。
- goroutine 优势：初始栈仅 **2KB**，按需增长（连续栈，扩容时复制到更大的栈）；创建/切换在用户态完成，开销远小于线程；可轻松创建数十万个。

### 2. make 与 new、数组与切片

- **new(T)**：分配 T 的零值内存，返回 `*T` **指针**，可用于任何类型。

- **make(T, args)**：只用于 **slice、map、channel**，初始化其内部数据结构，返回 **T 本身**（非指针）。
- **数组**：长度是类型的一部分（`[3]int` 和 `[4]int` 是不同类型），**值类型**，赋值和传参会整体拷贝。
- **切片**：对底层数组的引用视图，长度可变，传参拷贝的是切片头（指针、len、cap）。

### 3. for range 的坑

- range 表达式只**求值一次**；遍历数组时遍历的是**数组副本**，遍历切片时复制的是切片头。
- **Go 1.22 之前**：循环变量在整个循环中**只有一个实例**，每次迭代覆盖其值，取地址 `&v` 得到的都是同一地址；闭包/goroutine 捕获循环变量会全部看到最后一个值。
- **Go 1.22 起**：`for` 循环的变量**每次迭代都是新变量**，上述问题消失（go.mod 中 go 版本 ≥ 1.22 生效）。
- 遍历 map 时修改 value 的字段需通过 `m[k] = v` 写回（map 元素不可寻址）。

### 4. 字符串、rune 与拼接

- string 底层是**只读字节数组 + 长度**，不可变；`len(s)` 是**字节数**。
- **byte = uint8**，表示一个字节；**rune = int32**，表示一个 Unicode 码点；`for _, r := range s` 按 rune 遍历（UTF-8 解码）；统计字符数用 `utf8.RuneCountInString` 或 `len([]rune(s))`。
- 拼接方式与性能：`+`（每次生成新串，少量拼接可用）、`fmt.Sprintf`（反射，较慢）、`strings.Join`、`bytes.Buffer`、**`strings.Builder`（推荐，可 `Grow` 预分配，`String()` 零拷贝）**。
- string 与 []byte 互转一般会发生**内存拷贝**（编译器对部分场景有优化）。

### 5. defer、panic 与 recover

- defer 按 **LIFO（后进先出）** 顺序执行，在函数 return 赋值返回值之后、真正返回之前执行。
- defer 的**参数在注册时就求值**；若用闭包则引用的是变量最终值。
- defer 可以**修改命名返回值**：`return x` 实际是「先给返回值赋值 → 执行 defer → 返回」。
- 场景：释放资源（关闭文件、解锁 `mu.Unlock()`）、recover 捕获 panic、记录耗时。
- **recover 只在 defer 函数中直接调用才有效**，且只能捕获**当前 goroutine** 的 panic；子 goroutine 的 panic 未 recover 会导致整个进程崩溃。
- 性能：Go 1.14 引入**开放编码（open-coded）defer**，常见场景下开销接近直接调用。

### 6. 空结构体、tag、init 与格式化

- **struct{} 不占内存**（`unsafe.Sizeof` 为 0，所有零大小对象可能指向同一地址 zerobase）。用途：
  - 实现集合 `map[string]struct{}`；
  - 只传信号的 channel `chan struct{}`；

  - 只有方法没有字段的类型。
- **struct tag**：字段元信息，如 `` `json:"name,omitempty"` ``，通过**反射** `reflect.StructTag.Get` 读取，用于 JSON、ORM、参数校验。
- **init 执行顺序**：先初始化依赖包 → 本包**包级变量** → 本包 **init()**（一个包可以有多个 init，按文件和出现顺序执行）→ `main()`。init 不能被显式调用。
- **打印**：`%v` 默认格式；`%+v` 结构体带**字段名**；`%#v` 输出 **Go 语法表示**（含类型名）；`%T` 输出类型。

### 7. 函数传参、多返回值与 nil 比较

- **Go 只有值传递**：slice、map、channel、指针传递的是「描述符/指针」的副本，所以函数内修改元素可影响外部，但重新赋值（如 append 导致扩容）不影响外部变量。
- **多返回值实现**：Go 1.17 之前通过**栈**传递参数和返回值；Go 1.17 起（amd64 等平台）采用**基于寄存器的调用约定**。
- **两个 nil 可能不相等**：interface 由（类型, 值）组成，只有**类型和值都为 nil** 时接口才等于 nil。
  - 例：`var p *T = nil; var i interface{} = p` → `i != nil`（类型为 *T）。函数返回自定义 error 时要直接 `return nil`，不要返回值为 nil 的具体类型指针。
- **两个 interface 可以比较**：动态类型相同且动态值相等时相等；若动态类型不可比较（slice、map、func）则**运行时 panic**。

### 8. unsafe.Pointer 与 uintptr

- 普通指针 `*T`：类型安全，不能进行指针运算，不同类型指针不能直接转换。
- **unsafe.Pointer**：通用指针，可与任意类型指针互转，**GC 会追踪**它指向的对象。
- **uintptr**：一个**整数**，可做算术运算，但 **GC 不把它当引用**，对象可能被回收或因栈增长被移动。
- 常见用法：`unsafe.Pointer` ↔ `uintptr` 做偏移计算后**在同一表达式内**转回 `unsafe.Pointer`，不要把 uintptr 存在变量中再转回。

## 二、slice 与 map

### 9. slice 底层结构与扩容

- 结构 `runtime.slice`：`array`（指向底层数组的指针）、`len`、`cap`。
- **扩容规则（Go 1.18 起）**：
  1. 期望容量 > 2 倍旧容量 → 直接使用期望容量；
  2. 旧容量 **< 256** → 新容量翻倍；
  3. 否则循环 `newcap += (newcap + 3*256) / 4`，**从 2 倍平滑过渡到约 1.25 倍**；

4. 最后按**内存规格（size class）向上取整**，所以实际 cap 可能比计算值略大。

- Go 1.18 之前：旧容量 < 1024 翻倍，否则每次增加 25%。
- 扩容会分配**新数组并拷贝**，原切片不受影响。

### 10. slice 共享底层数组的陷阱

- 切片截取 `s2 := s1[1:3]` 与原切片**共享底层数组**，修改 s2 元素会影响 s1。
- 对 s2 `append`：若未超过 cap，**会覆盖 s1 中对应位置的数据**；超过 cap 则扩容到新数组，此后两者互不影响。
- 可用**三索引切片** `s1[1:3:3]` 限制 cap，使 append 必然触发拷贝。
- 函数传 slice：修改元素影响外部；函数内 append 后外部变量的 len 不变，需要**返回新切片**或传 `*[]T`。
- 大数组截取小切片会导致整个大数组无法释放 → 用 `copy` 拷贝出需要的部分。

### 11. map 底层实现（经典 hmap 结构，Go 1.24 之前）

- `hmap`：`count`、`B`（桶数量为 2^B）、`buckets` 指针、`oldbuckets`（扩容时旧桶）、`hash0`（哈希种子）、`nevacuate`（迁移进度）。
- 桶 `bmap`：每个桶存 **8 个 key/value**，`tophash[8]` 存哈希值**高 8 位**用于快速比较；key 和 value 分别连续存放（减少内存对齐浪费）；满了通过 **overflow 溢出桶**链接。
- 查找：hash(key) → **低 B 位选桶** → 用**高 8 位**比较 tophash → 再比较完整 key → 桶内找不到则查溢出桶。
- **Go 1.24 起**内置 map 改为 **Swiss Table** 实现（以 8 个槽为一组，带控制字节，可用 SIMD/位运算批量匹配），面试仍以经典 hmap 结构为主，可补充说明。

### 12. map 扩容

- 触发时机（插入/修改时检查）：
  - **负载因子 > 6.5**（count / 2^B）→ **翻倍扩容**（B+1）；
  - **溢出桶过多**（B < 15 时溢出桶数 ⩾ 2^B；B ⩾ 15 时 ⩾ 2^15）→ **等量扩容**，重新整理使数据更紧凑（常见于大量删除后）。
- 过程：**渐进式迁移**——分配新桶，旧桶挂到 oldbuckets，之后每次**写入或删除**时迁移当前访问的桶并额外迁移一个桶，直到全部完成；翻倍扩容时旧桶数据按 hash 新增的那一位分流到 `原下标` 或 `原下标 + 旧桶数`。

### 13. map 使用要点

- **遍历无序**：runtime 每次遍历**随机选择起始桶和桶内起始位置**，有意避免开发者依赖顺序（扩容后顺序本来也会变）。需要有序：取出 key 排序后再按 key 访问。
- **并发不安全**：并发读写会触发 `fatal error: concurrent map read and map write`（无法 recover）。解决：`sync.RWMutex` 加锁、`sync.Map`、分片加锁 map。

- **key 必须可比较**：slice、map、func 不能做 key（需要 `==` 判断相等）；浮点数可以但 NaN 不等于自身，不建议使用。
- **不能对 map 元素取地址**：`&m[k]` 编译报错，扩容时元素会被迁移，地址不稳定。
- **删除 key 不会释放桶内存**：只标记槽位为空，map 不会缩容；大量删除后需重建新 map 才能释放内存。
- **可以边遍历边删除**：语言规范允许，被删除的未遍历元素不会再出现；遍历中新增的元素可能出现也可能不出现。

## 三、channel 与 select

### 14. CSP 与 channel 底层结构

- **CSP（通信顺序进程）**：「**不要通过共享内存来通信，而要通过通信来共享内存**」，goroutine 之间通过 channel 传递数据。
- `hchan` 结构：
  - `buf`：**环形队列**（有缓冲 channel），`qcount` 元素个数、`dataqsiz` 容量、`sendx` / `recvx` 发送和接收下标；
  - `sendq` / `recvq`：阻塞的发送者/接收者**等待队列**（`sudog` 双向链表）；
  - `lock`：互斥锁，保护整个结构；
  - `closed`：关闭标志；`elemtype`、`elemsize`。
- channel 由 `makechan` 在**堆上**分配。

### 15. 发送与接收流程

- **发送 `ch <- v`**（加锁后）：
  1. `recvq` 有等待的接收者 → **直接把数据拷贝给它**并唤醒，不经过缓冲区；
  2. 缓冲区未满 → 数据拷贝到 `buf[sendx]`；
  3. 否则 → 当前 goroutine 封装为 sudog 加入 `sendq`，**gopark 挂起**，等待被接收者唤醒。
- **接收 `v := <-ch`**（加锁后）：
  1. `sendq` 有等待的发送者：无缓冲 → 直接从发送者拷贝；有缓冲（此时缓冲区必满）→ 取队头数据，并把发送者的数据放入队尾，唤醒发送者；
  2. 缓冲区有数据 → 从 `buf[recvx]` 取出；
  3. 否则 → 加入 `recvq` 挂起。
- 无缓冲 channel 是**同步**的：发送和接收必须同时就绪。

## 16. channel 操作结果总结

| 操作 | nil channel | 已关闭 channel | 正常 channel |
| --- | --- | --- | --- |
| 发送 | 永久阻塞 | **panic** | 阻塞或成功 |
| 接收 | 永久阻塞 | 先读完缓冲区剩余数据，之后返回**零值, false** | 阻塞或成功 |
| 关闭 | **panic** | **panic** | 成功 |

- 关闭 channel 会唤醒所有等待的接收者（得到零值）和发送者（发送者会 panic）。
- 用 `v, ok := <-ch` 判断是否已关闭；`for v := range ch` 在关闭且读完后退出。
- **关闭原则**：由**发送方**关闭，**不要在接收端关闭**，多个发送者时通过额外的信号 channel 或 `sync.Once` 协调。

## 17. channel 导致的 goroutine 泄漏

- 场景：
  - 向无人接收的 channel 发送（如超时返回后，worker 仍向无缓冲 channel 发送结果）；
  - 从永远没有数据也不会关闭的 channel 接收；
  - `for range ch` 但发送方忘记 close；
  - 操作 nil channel。
- 预防：使用**带缓冲 channel**（容量 1 存结果）、`select` + `ctx.Done()` 退出、确保发送方关闭 channel；用 `runtime.NumGoroutine`、pprof `goroutine` 剖面或 `goleak` 排查。

## 18. select 机制与实现

- 每个 case 必须是 channel 操作；**多个 case 同时就绪时随机选择一个**，防止饥饿；都不就绪时有 `default` 走 default（非阻塞），否则阻塞。
- 特例：空 `select {}` **永久阻塞**；只有一个 case 编译器改写为普通收发；一个 case + default 改写为非阻塞收发（`selectnbsend` / `selectnbrecv`）。
- 运行时 `runtime.selectgo`：
  1. 生成**随机轮询顺序 pollorder** 和按 channel 地址排序的**加锁顺序 lockorder**（避免死锁）；
  2. 按 lockorder 锁定所有 channel，按 pollorder 查找就绪 case，找到则执行并返回；
  3. 无就绪且无 default → 当前 goroutine 的 sudog 加入**所有** channel 的等待队列并挂起；
  4. 被某个 channel 唤醒后，从其他 channel 的等待队列中移除自己。
- 常用：`select` + `time.After` 实现超时；`for-select` + `ctx.Done()` 实现可取消循环（循环中频繁使用 `time.After` 应改为复用 `time.Timer`）。

## 四、sync 与并发原语

### 19. 安全读写共享变量的方式

- **sync.Mutex / RWMutex** 互斥锁、读写锁；
- **channel** 串行化访问（由单个 goroutine 持有数据）；
- **sync/atomic** 原子操作（`atomic.Int64`、`atomic.Value`、CAS）；
- `sync.Once`、`sync.Map`、`sync.Pool` 等；
- 并发安全检测：`go test -race` / `go run -race`。

### 20. 原子操作与锁

- 原子操作由 **CPU 指令**保证（x86 上如 `LOCK` 前缀指令、`XCHG`、`CMPXCHG`），不可被中断。
- 区别：
  - 原子操作针对**单个变量**，无上下文切换，性能高，属于乐观/无锁思想（CAS 失败重试）；
  - 锁可保护**一段临界区**的多个操作，竞争激烈时 goroutine 会休眠，属于悲观思想。
- Go 1.19 起提供 `atomic.Int32`、`atomic.Int64`、`atomic.Bool`、`atomic.Pointer[T]` 等类型化原子类型。

### 21. Mutex 实现与两种模式

- 结构：`state int32`（低位依次为 **locked、woken、starving 标志，其余位为等待者数量**）+ `sema` 信号量。
- 加锁：快速路径 **CAS** 将 state 从 0 改为 locked；失败进入慢路径，满足条件时**自旋**（多核、GOMAXPROCS > 1、有其他运行中的 P 且本地队列为空、自旋次数 < 4），否则通过信号量休眠排队。
- **正常模式**：等待者按 FIFO 排队，但被唤醒的 goroutine 要与**新到来的 goroutine 竞争**锁，新来者在 CPU 上运行、更容易抢到 → 吞吐高但可能让队列中的等待者饥饿。
- **饥饿模式**：等待者等待**超过 1ms** 时切换；锁**直接交给队头等待者**，新来的 goroutine 不自旋、不抢锁，直接排到队尾。当获得锁的等待者是队列最后一个或等待时间 < 1ms 时切回正常模式。
- 自旋时间很短且有次数限制，不会长期占用 CPU。
- Mutex **不可重入**，同一 goroutine 重复加锁会死锁；不要复制已使用的 Mutex（`go vet` 可检查）。
- **RWMutex**：读锁共享、写锁独占；有写锁等待时新的读锁会阻塞，**防止写饥饿**；适合读多写少。

### 22. sync.Once 与 WaitGroup

- **sync.Once**：保证函数只执行一次（单例、懒加载配置）。

  - 结构：`done`（原子标志）+ `m Mutex`；
  - 快速路径**原子读** done == 1 直接返回；慢路径加锁后**双重检查**，执行 f 后再将 done 原子置 1（f 执行完才置位，保证其他 goroutine 返回时 f 已完成）；
  - f 中 panic 也视为已执行；f 内再调用同一个 Once 的 Do 会死锁。Go 1.21 新增 `sync.OnceFunc`、`OnceValue`。
- **sync.WaitGroup**：等待一组 goroutine 完成。
  - 内部一个原子状态：**计数器（高 32 位）+ 等待者数量（低 32 位）**，配合信号量；
  - `Add(n)` 增加计数（应在启动 goroutine **之前**调用），`Done()` 即 `Add(-1)`，计数为 0 时唤醒所有 `Wait` 的 goroutine；计数变为负数会 panic；
  - Go 1.25 新增 `wg.Go(func)`，自动完成 Add 与 Done。

### 23. sync.Map

- 适用场景（官方说明）：**key 只写一次读多次（只增长的缓存）**，或**多个 goroutine 读写互不相交的 key 集合**。写多、key 频繁变化时不如 `map + RWMutex`。
- 经典实现（Go 1.24 之前）：**read map（只读，原子访问，无锁）+ dirty map（加锁访问）+ misses 计数**。
  - **Load**：先查 read；未命中且 `amended=true`（dirty 中有 read 没有的 key）时加锁查 dirty，misses++；**misses ⩾ len(dirty) 时把 dirty 提升为 read**；
  - **Store**：key 在 read 中且未被标记 expunged → 直接 CAS 更新 entry；否则加锁写入 dirty（dirty 为 nil 时先从 read 复制未删除的条目）；
  - **Delete**：read 中存在则把 entry 置为 nil（**延迟删除**）。
- **nil 与 expunged 两种删除状态**：nil 表示已删除但 dirty 中可能仍有该 key；expunged 表示已删除**且 dirty 中没有**，复制 read → dirty 时把 nil 标记为 expunged 并跳过，之后对该 key 的写入必须加锁同时写入 dirty，保证 dirty 提升为 read 时数据完整。
- Go 1.24 起 sync.Map 底层改为并发哈希字典树（HashTrieMap）实现，API 不变。

## 五、context

### 24. context 的作用与接口

- 作用：在 goroutine 调用链之间传递**取消信号、超时/截止时间、请求域数据**，典型用于 HTTP 请求、RPC、数据库调用的超时控制与级联取消。
- `Context` 接口四个方法：`Deadline()`、`Done()`（返回只读 channel，取消时关闭）、`Err()`（`Canceled` 或 `DeadlineExceeded`）、`Value(key)`。
- 创建：根 `context.Background()` / `context.TODO()`；派生 `WithCancel`、`WithTimeout`、`WithDeadline`、`WithValue`；Go 1.20 `WithCancelCause`，Go 1.21 `WithoutCancel`、`AfterFunc`。

### 25. 取消传播与 Value 查找

- **树形结构**：父 context 取消时，会**递归取消所有子 context**（cancelCtx 维护 children 集合）；子取消不影响父。
- 取消过程：关闭 `done` channel、设置 err、遍历取消子节点、从父节点的 children 中移除自己。`WithTimeout` 基于 `time.AfterFunc` 定时器到期自动调用 cancel。
- **Value 查找**：valueCtx 每个节点只存一个 k/v，查找时**沿父链向上逐层查找**，O(n)，找不到返回 nil。
- 最佳实践：
  - 作为函数**第一个参数** `ctx context.Context`，不要放进结构体；
  - **WithCancel/WithTimeout 返回的 cancel 必须调用**（`defer cancel()`），否则定时器和子节点资源泄漏；
  - Value 只传**请求域数据**（traceID、用户信息），key 使用自定义私有类型避免冲突，不要传可选业务参数；
  - 不要传 nil context，不确定时用 `context.TODO()`。

## 六、interface 与反射

### 26. interface 底层：eface 与 iface

- **eface**（空接口 `interface{}` / `any`）：`_type`（动态类型信息）+ `data`（指向数据的指针）。
- **iface**（带方法的接口）：`tab *itab` + `data`。`itab` 包含**接口类型、动态类型、方法表 fun**（按方法名排序的函数指针），itab 会被全局缓存。
- **隐式实现（鸭子类型）**：类型实现了接口的全部方法就自动实现该接口，可用 `var _ I = (*T)(nil)` 在编译期校验。
- **方法集规则**：值类型 T 的方法集只含值接收者方法；指针 `*T` 的方法集包含值和指针接收者方法 → 用指针接收者实现接口时，只有 `*T` 实现了该接口。

### 27. 类型转换与类型断言

- **类型转换** `T(x)`：编译期在兼容类型间转换（如 `int64(i)`、`[]byte(s)`）。
- **类型断言** `x.(T)`：只能用于**接口值**，运行时检查动态类型；`v, ok := x.(T)` 失败时 ok 为 false，单返回值形式失败会 panic。
- **type switch**：`switch v := x.(type) { case int: ... }`。
- interface 应用：依赖抽象便于 mock 测试、标准库 `io.Reader/Writer`、`error`、`sort.Interface`、多态与插件化。

### 28. 反射

- 基于 interface 的类型信息，在**运行时**获取和修改变量的类型与值：`reflect.TypeOf()` 返回 `reflect.Type`，`reflect.ValueOf()` 返回 `reflect.Value`。
- **反射三大定律**：接口值 → 反射对象；反射对象 → 接口值（`v.Interface()`）；**要修改反射对象，值必须可设置**（传指针并用 `Elem()`，`CanSet()` 为 true）。
- 应用：`encoding/json` 序列化、ORM 映射、struct tag 解析、依赖注入、`fmt` 打印。
- 缺点：性能较差、失去编译期类型检查、代码可读性差。
- 比较两个对象完全相同：`reflect.DeepEqual`（递归比较，可比较 slice、map）；可比较类型直接用 `==`。

## 七、GMP 调度模型

### 29. GMP 模型

- **G（Goroutine）**：协程，包含栈、程序计数器、状态等。
- **M（Machine）**：**操作系统线程**，真正执行代码，必须绑定 P 才能执行 G（默认最多 10000 个，可用 `debug.SetMaxThreads` 调整）。
- **P（Processor）**：逻辑处理器，持有**本地运行队列**（最多 **256** 个 G）和 `runnext`（下一个优先运行的 G）、mcache 等资源；数量由 **GOMAXPROCS** 决定（默认等于 CPU 核数；Go 1.25 起在容器中会考虑 cgroup CPU 限制）。
- **全局运行队列**：本地队列满时，一半 G 被移到全局队列，访问全局队列需要加锁。
- 调度器（scheduler）作用：把大量 G 高效地分配到少量 M 上执行，实现 M:N 调度。

### 30. 调度策略

- **work stealing（工作窃取）**：P 的本地队列为空时，先查全局队列，再从其他 P 的本地队列**偷一半** G。
- **hand off（移交）**：G 执行**阻塞系统调用**时，M 与 P 解绑，P 交给其他空闲 M（或新建 M）继续运行其他 G；系统调用返回后 G 尝试获取空闲 P，获取不到则放入全局队列。
- **抢占**：
  - Go 1.14 之前：**基于协作的抢占**，在函数调用时检查抢占标志，纯计算死循环无法被抢占；
  - **Go 1.14 起：基于信号的异步抢占**，sysmon 发现 G 运行超过 **10ms**，向其所在 M 发送 **SIGURG** 信号强制抢占。
- **sysmon 监控线程**：不需要绑定 P，负责抢占长时间运行的 G、回收系统调用中阻塞的 P、netpoll 检查、触发强制 GC。

### 31. 调度时机与查找可运行 G 的顺序

- 调度时机：

    - 主动让出：`runtime.Gosched()`；
    - **阻塞**：channel 收发、锁、`time.Sleep`、网络 IO（netpoller）等调用 `gopark`；
    - 被抢占：运行超过 10ms；
    - 系统调用；G 执行结束。
- M 寻找可运行 G（`findRunnable`）：
    1. 每调度 **61 次**优先查一次**全局队列**（防止全局队列饥饿）；
    2. 本地队列（先 `runnext`）；
    3. 全局队列（批量取一部分）；
    4. **netpoll** 获取 IO 就绪的 G；
    5. 从其他 P **窃取**；
    6. 都没有则 M 休眠。

### 32. 为什么需要 P、g0 与 m0

- **早期 GM 模型（Go 1.1 前）的问题**：所有 G 放在**全局队列，需全局锁**，竞争激烈；G 在不同 M 间频繁切换，局部性差；每个 M 都持有内存缓存，浪费内存；系统调用阻塞时线程切换频繁。
- 引入 P 后：**本地队列无锁访问**、内存缓存 mcache 挂在 P 上（数量 = GOMAXPROCS），并支持 work stealing 和 hand off。
- P 在程序启动时按 GOMAXPROCS 创建；M 在没有足够的 M 关联 P 运行可运行 G 时创建（如系统调用阻塞时）。
- **m0**：进程启动的**第一个线程**，负责初始化运行时和启动第一个 G（main goroutine），之后与普通 M 一样。
- **g0**：每个 M 都有一个 g0，**专门负责调度**（执行 schedule、创建 G、栈扩容、GC 辅助等运行时代码），使用固定大小的系统栈而非可增长的协程栈；用户 G 需要调度时先切换到 g0 栈（`mcall` / `systemstack`），选出下一个 G 后再切换到其用户栈（`gogo`）。

## 八、内存管理与 GC

### 33. 内存分配

- 借鉴 **TCMalloc**，多级缓存：
    - **mcache**：每个 P 一个，**无锁**分配；
    - **mcentral**：每种规格一个，mcache 不足时从这里加锁获取 span；
    - **mheap**：全局堆，管理页（8KB），向操作系统申请内存。
- **mspan**：由连续页组成，按对象大小分为约 **68 种规格（size class）**。
- 按对象大小分配：

- **微对象**（< 16B 且不含指针）：tiny 分配器合并到同一 16B 块；
- **小对象**（16B ~ 32KB）：按 size class 从 mcache 对应 span 分配；
- **大对象**（> 32KB）：直接从 mheap 分配。

### 34. 内存逃逸

- **逃逸分析**：编译器决定变量分配在**栈**（函数返回自动回收，开销小）还是**堆**（由 GC 回收）。查看：`go build -gcflags="-m"`。
- 常见逃逸场景：
  - 函数**返回局部变量的指针**；
  - 变量赋值给 **interface 类型**（如传给 `fmt.Println` 的参数）；
  - **闭包**引用外部变量；
  - **栈空间不足**/对象过大，或切片长度在运行时才确定（`make([]int, n)`）；
  - 向 channel 发送指针、指针存入 slice/map。
- 影响：堆分配更慢，并**增加 GC 压力**。优化：小对象尽量值传递、预分配容量、避免不必要的 interface 和指针、使用 `sync.Pool` 复用对象。

### 35. 内存泄漏场景与排查

- 常见原因（Go 有 GC 仍会泄漏，因为对象**仍被引用**）：
  - **goroutine 泄漏**（最常见）：channel 阻塞、无退出条件的循环、锁未释放；
  - 子切片/子字符串引用大数组；
  - 全局 map / 缓存只增不删；
  - `time.Ticker` 未 Stop（Go 1.23 起未引用的 Timer/Ticker 可被 GC 回收）；
  - 未关闭的 HTTP response body、文件等资源。
- 排查：引入 `net/http/pprof`，`go tool pprof` 分析 **heap**（内存）和 **goroutine** 剖面，对比两个时间点的差异；监控 `runtime.NumGoroutine()`；测试中使用 `go.uber.org/goleak`。

### 36. 三色标记法与 GC 根对象

- Go GC：**并发三色标记清除**，非分代、非整理（不移动对象）。
- 三色：
  - **白色**：未被访问，标记结束后仍为白色的对象会被回收；
  - **灰色**：已被访问，但其引用的对象还未扫描完；
  - **黑色**：自身及引用的对象都已扫描。
- 过程：根对象置灰 → 不断从灰色集合取出对象，将其引用对象置灰、自身置黑 → 灰色集合为空时回收所有白色对象。
- **根对象**：全局变量（数据段、BSS 段）、每个 goroutine **栈上的变量**、寄存器中的指针。

- **STW（Stop The World）**：暂停所有用户 goroutine，Go 的 STW 通常在亚毫秒级，只发生在标记开始和标记终止阶段。

### 37. 写屏障与混合写屏障

- 并发标记的难点：用户程序同时修改引用，可能导致**存活对象被误回收**。发生条件（**三色不变式被破坏**）：
  1. 黑色对象引用了白色对象；
  2. 同时灰色对象到该白色对象的路径被删除。
- **强三色不变式**：黑色对象不能直接引用白色对象 → **插入写屏障**（Dijkstra）：新增引用时把被引用对象置灰。缺点：栈上不开启写屏障，标记结束需 **STW 重新扫描栈**。
- **弱三色不变式**：黑色引用的白色对象必须有灰色对象可达保护 → **删除写屏障**（Yuasa）：删除引用时把被删除的对象置灰。缺点：回收精度低，需要起始时快照栈。
- **混合写屏障（Go 1.8）**：结合两者——
  - GC 开始时**将栈上可达对象全部标记为黑色**（逐个 goroutine 扫描，无需整体 STW，之后**无需重新扫描栈**）；
  - GC 期间栈上**新创建的对象为黑色**；
  - 堆上**被删除的引用对象和被添加的引用对象都标记为灰色**。
  - 大幅缩短 STW 时间。

### 38. GC 流程、触发时机与调优

- **流程**：
  1. **清扫终止**（STW）：完成上一轮清扫，准备开启写屏障；
  2. **并发标记**：开启写屏障，后台标记 worker 约占用 **25% CPU**，分配内存过快的 goroutine 需进行**辅助标记（mark assist）**；
  3. **标记终止**（STW）：关闭写屏障，计算下次触发目标；
  4. **并发清扫**：回收白色对象，可在分配内存时惰性清扫。
- **触发时机**：
  - 堆内存达到目标值：**GOGC（默认 100）**表示新分配内存达到上次 GC 后存活堆的 100% 时触发；
  - **定时触发**：超过 **2 分钟**未 GC 时 sysmon 强制触发；
  - 手动调用 `runtime.GC()`；
  - 接近 **GOMEMLIMIT**（Go 1.19 引入的软内存上限）时会更积极地 GC。
- **关注指标**：CPU 占用率、STW 停顿时间、GC 频率、堆内存峰值。
- **调优**：

    - **减少堆分配**（最有效）：预分配 slice/map 容量、`sync.Pool` 复用对象、避免逃逸、`strings.Builder` 拼接；
    - 调整 **GOGC**（调大降低 GC 频率但占用更多内存）；
    - 设置 **GOMEMLIMIT** 防止 OOM，容器环境下常配合使用。
- **观察**：`GODEBUG=gctrace=1`、`runtime.ReadMemStats`、pprof（heap / allocs）、`go tool trace`。
- 补充：Go 1.25 以实验特性（`GOEXPERIMENT=greenteagc`）引入新的「Green Tea」GC，优化小对象标记的内存局部性。

## 九、常见并发模式与代码题

### 39. 常见并发模式

- **Worker Pool**：固定数量 goroutine 从任务 channel 取任务，控制并发度。
- **Pipeline 流水线**：每个阶段一个 goroutine，通过 channel 串联，上游关闭 channel 通知下游。
- **Fan-out / Fan-in**：多个 goroutine 并行处理同一输入（扇出），再合并到一个 channel（扇入）。
- **errgroup**（`golang.org/x/sync/errgroup`）：并发执行一组任务，任一出错可通过 ctx 取消其余任务，`SetLimit` 限制并发数。
- **singleflight**：合并相同 key 的并发请求，防缓存击穿。
- **限流**：带缓冲 channel 作信号量；`golang.org/x/time/rate` 令牌桶；`time.Ticker` 匀速。
- **超时控制**：`context.WithTimeout` / `select` + `time.After`。
- **优雅退出**：监听 `os.Signal`，`http.Server.Shutdown(ctx)`，`WaitGroup` 等待在途任务结束。

### 40. 并发控制 + 超时（代码模板）

- 需求：1000 个任务，最多 10 个并发，整体超时 1 秒。

```
func run(tasks []func(ctx context.Context) error) error {
    ctx, cancel := context.WithTimeout(context.Background(), time.Second)
    defer cancel()

    g, ctx := errgroup.WithContext(ctx)
    g.SetLimit(10) // 最多 10 个 goroutine 同时运行
    for _, t := range tasks {
        g.Go(func() error { return t(ctx) }) // Go 1.22+ 循环变量每次迭代独立
    }
    return g.Wait() // 任一任务出错或超时，ctx 被取消
}
```

- 不依赖第三方库时：`sem := make(chan struct{}, 10)`，启动前 `sem <- struct{}{}`、结束时 `<-sem`，配合 `WaitGroup` 与 `select { case <-ctx.Done(): }`。

### 41. 交替打印（代码模板）

- 需求：两个 goroutine 交替打印奇偶数 1~10（思路可推广到三个 goroutine 循环打印 abc：用 N 个 channel 形成环）。

```go
func main() {
    odd, even := make(chan struct{}), make(chan struct{})
    var wg sync.WaitGroup
    wg.Add(2)
    go func() { // 打印奇数
        defer wg.Done()
        for i := 1; i <= 10; i += 2 {
            <-odd
            fmt.Println(i)
            even <- struct{}{}
        }
    }()
    go func() { // 打印偶数
        defer wg.Done()
        for i := 2; i <= 10; i += 2 {
            <-even
            fmt.Println(i)
            if i < 10 {
                odd <- struct{}{}
            }
        }
    }()
    odd <- struct{}{} // 启动信号
    wg.Wait()
}
```

- 要点：用**无缓冲 channel 传递「令牌」**保证顺序；注意最后一轮不要再发送令牌，否则会因无人接收而死锁。也可用 `sync.Cond` 或 `sync.Mutex` + 共享计数实现。